Deep variational Gaussian process networks in geoML: the second moment, realizations that carry it, and what the code does

Prepared for Ítalo Gomes Gonçalves
branch `deep-second-moment`, commits `c6d707c`–`1b84ed5`

19 October 2026

Abstract

geoML builds spatial models as networks of variational Gaussian process (GP) nodes. A node whose input is another node's uncertain output — a deep layer, or a `GPWalk` — reports moments that, until this work, left out the spread of its mean over the input — 16–61% of the deep model's variance on the folded section — and its realizations carried still less: 25–59% of the variance they were reported with, 6% on a shallow GP with three experts. This report sets out the theory of the node as geoML implements it (the whitened variational posterior, the experts, the expected kernel), then the three pieces added on the branch: (i) the *second moment* at uncertain inducing inputs, by a quadrature over the input with the inducing inputs conditioned on each node; (ii) *pathwise realizations*, every expert drawing from one random-feature prior, with Matheron's rule supplying what the inducing points leave out and an exact rescale making the blend hold the reported variance; and (iii) a prediction that integrates the realizations' own residual analytically. Each equation is tied to the function that computes it. The last sections report the measurements: the gates, training on the folded section, Walker Lake and chapter 16, two regressions traced to their commits, and a profile of where the time goes.

Contents

1 The variational GP node **3**
1.1 Kernel and inputs . . . 3
1.2 Inducing points and the whitened posterior . . . 3
1.3 Moments at a certain input . . . 3
1.4 The bound . . . 3
1.5 Experts . . . 4

2 Deep networks and the expected kernel **4**
2.1 Chains . . . 4
2.2 The expected kernel . . . 4
2.3 The second moment (Girard) . . . 4
2.4 The walk . . . 5

3 Realizations that hold the reported variance **5**
3.1 What the realizations used to carry . . . 5
3.2 Random-feature prior draws . . . 6
3.3 Each expert's conditioned draw, from one prior . . . 6
3.4 The blend and its exact rescale . . . 6
3.5 Through the parent: readers of uncertain inputs . . . 7
3.6 Location-keyed normals . . . 7
3.7 The prediction integrates the residual (2(b)) . . . 7

4 Training **7**

5 Where each piece lives **8**

6 Measurements **8**
6.1 The realizations hold the reported variance 8
6.2 Training on the second moment 9
6.3 Two regressions, traced 9
6.4 Where the time goes 11

1 The variational GP node

1.1 Kernel and inputs

A GP node `BasicGP` reads coordinates $x \in \mathbb{R}^d$ from its parent and models a latent function $f : \mathbb{R}^d \to \mathbb{R}^s$ (s outputs, `size`) whose components are independent GPs with unit variance and a common correlation k. Distances are measured in *ranges*: with ρ_j the range of dimension j, the scaled distance is $\delta^2 = \sum_j (x_j - y_j)^2/\rho_j^2$. Every kernel the deep machinery accepts is written as a positive mixture of Gaussians in δ (`_kernel_components`):

$$k(x,y) \;=\; c + \sum_{q=1}^{Q} w_q \exp\Big(-\omega_q\, \delta^2\Big), \qquad w_q > 0,\; c \geq 0. \tag{1}$$

The Gaussian kernel is one component, $\omega_1 = 3$, $w_1 = 1$, so that the range is the practical range ($k = e^{-3}$ at $\delta = 1$). The Matérn family and the exponential are tables of eight fitted Gaussians each (`_KERNEL_MIXTURES`, weights summing to one, within 5×10^{-4} of the exponential); the rational quadratic is 48 components placed about its trained scale. Spherical and cubic kernels are not mixtures and are refused where an input is uncertain.

1.2 Inducing points and the whitened posterior

The root node holds m inducing inputs Z per expert (`BasicInput`, `inducing_points`); every GP node above it sees the same Z carried through the network. With $K = k(Z,Z) + \epsilon I$ and its Cholesky factor L ($K = LL^\top$), the node's variational posterior over the inducing values $u = f(Z)$ is, per output,

$$q(u) = \mathcal{N}(\mu, S), \qquad \mu = L\,\alpha_w, \qquad S = \Big(K^{-1} + D^{-1}\Big)^{-1} = K - K(K+D)^{-1}K, \tag{2}$$

with α_w the free `alpha_white_i` and $D = \mathrm{diag}(\delta_1, \ldots, \delta_m)$ the positive `delta_i`. The covariance is that of a prior conditioned on pseudo-observations of noise D; the mean is free. In the code $\alpha = K^{-1}\mu$ (`alpha`), $(K+D)^{-1}$ is `cov_smooth_inv`, and the square root `chol_r`

$$R = L^{-\top}\,\mathrm{chol}(W), \qquad W = (I + L^\top D^{-1} L)^{-1}, \qquad RR^\top = K^{-1} - (K+D)^{-1} \tag{3}$$

is formed without the difference of inverses (`_whitened_root`), which cancels catastrophically on an ill-conditioned set. Note $KRR^\top K = S$.

1.3 Moments at a certain input

For a certain location x, with $\mathbf{k} = k(x, Z)$ and a per-expert bias b,

$$m(x) = \mathbf{k}\,\alpha + b, \tag{4}$$

$$v(x) = 1 - \mathbf{k}(K+D)^{-1}\mathbf{k}^\top \;=\; \underbrace{1 - \mathbf{k}K^{-1}\mathbf{k}^\top}_{\text{unexplained}} \;+\; \underbrace{\mathbf{k}RR^\top\mathbf{k}^\top}_{\text{explained draw}}\,. \tag{5}$$

The code reports the *explained* variance $\mathbf{k}(K+D)^{-1}\mathbf{k}^\top = 1 - v$ (`explained_var`), which weights the experts.

1.4 The bound

Training maximizes $\mathcal{L} = \sum_i \mathbb{E}_q[\log p(y_i \mid f(x_i))] - \sum \mathrm{KL} + \log p(\theta)$. Per expert and output (`expert_kl_terms`),

$$\mathrm{KL}(q(u)\,\|\,p(u)) = \tfrac{1}{2}\Big(-\mathrm{tr}\,[(K+D)^{-1}K] + \|\alpha_w\|^2 + \log|K+D| - \textstyle\sum_i \log\delta_i \Big), \tag{6}$$

which is the Gaussian KL with (2) substituted ($\operatorname{tr}(K^{-1}S) = m - \operatorname{tr}[(K+D)^{-1}K]$, $\log|S| = \log|K| + \log|D| - \log|K+D|$, the constants cancelling). The log-prior $\log p(\theta)$ holds the MAP priors of 0.6.5 (a Gamma on ranges with mode one, Gaussians on `Linear` weights). The data term is a Gauss–Hermite quadrature over the latent marginal for Gaussian leaves, and Monte Carlo over latent samples otherwise (section 4).

1.5 Experts

An input may be split into E overlapping *experts*, each a set of inducing points with its own $q(u_e)$, kernel matrices and bias. At a location the experts' moments are blended (`_blend`, `get_expert_weights`) with weights

$$\tilde{w}_e = \frac{1 - v_e}{v_e + 10^{-6}} + 10^{-6}, \qquad w_e = \tilde{w}_e \Big/ \textstyle\sum_f \tilde{w}_f, \qquad \bar{m} = \textstyle\sum_e w_e m_e, \quad \bar{v} = \textstyle\sum_e w_e v_e, \tag{7}$$

so an expert counts in proportion to how much of the prior its inducing points explain there. The blended variance $\sum_e w_e v_e$ is not the variance of $\sum_e w_e f_e$ unless the experts' functions agree; this gap is what section 3.4 closes for the realizations.

2 Deep networks and the expected kernel

2.1 Chains

Operation nodes (`Linear`, `Concatenate`, `SelectInput`, `Add`, ...) carry moments row by row. A GP node g reading an uncertain input $h(x) \sim \mathcal{N}(\mu_h, \Sigma_h)$ (diagonal) also needs the covariance between $h(x)$ and its own inducing inputs $h(Z)$, which are themselves the parent's prediction at Z, uncertain and correlated with $h(x)$. Under `GPOptions(propagation="joint")`, the default since 0.9.0, every node hands up a *chain* per expert: `_Joint(mean, variance, covariance)`, the covariance $[n, m, s]$ between the output at the data and at that expert's inducing points. For a GP node it is the posterior's own (`joint_gain`):

$$\operatorname{Cov}\left(f(x), f(Z)\right) = \mathbf{k}\,(K+D)^{-1}D. \tag{8}$$

2.2 The expected kernel

Reading an uncertain input, a GP node replaces $k(h(x), h(z))$ by its expectation. For one Gaussian component, inputs with independent coordinates of variances v_x, v_z per dimension, in range units ($a_j = (\mu_{x,j} - \mu_{z,j})^2/\rho_j^2$, $b_j = (v_{x,j} + v_{z,j})/\rho_j^2$):

$$\mathbb{E}\big[e^{-\omega\delta^2}\big] = \prod_j (1 + 2\omega b_j)^{-1/2} \exp\Big(-\omega \sum_j \frac{a_j}{1 + 2\omega b_j}\Big), \tag{9}$$

summed over (1) (`_expected_kernel`, `_mixture_terms`). The matrix $\mathbb{E}[K]$ over correlated uncertain inducing points stays positive definite because each Gaussian component does. This is the *first* moment of the kernel; it gives the posterior mean averaged over the input, but not the variance of that mean.

2.3 The second moment (Girard)

Averaging (4)–(5) over $h \sim \mathcal{N}(\mu_h, \Sigma_h)$ gives the mixture's moments

$$\bar{m} = \mathbb{E}_h[\mathbf{k}]\,\alpha + b, \qquad \bar{v} = \underbrace{\mathbb{E}_h\big[v(h)\big]}_{\text{conditional}} + \underbrace{\operatorname{Var}_h\left[\mathbf{k}\alpha\right]}_{\text{spread}}. \tag{10}$$

With $L_2 = \mathbb{E}_h[\mathbf{k}^\top \mathbf{k}]$ and $l = \mathbb{E}_h[\mathbf{k}]$, $\operatorname{Var}_h[\mathbf{k}\alpha] = \alpha^\top L_2 \alpha - (l\alpha)^2$ and $\mathbb{E}_h[v] = 1 - \operatorname{tr}[(K+D)^{-1}L_2]$. Where the inducing points are *certain* the code takes L_2 in closed form for the Gaussian kernel (`_second_moment`, pairs of Gaussians) and by 64 quadrature nodes otherwise (`_at_input_nodes`).

New: uncertain inducing inputs. A GP on a GP has uncertain inducing inputs correlated with the input, and until this work its moments omitted the spread. `_conditioned_sums` now integrates over $q = 64$ nodes ξ_t (`_input_nodes`: 32 scrambled Sobol points through the normal quantile and their negatives, whitened to unit covariance, so the rule is exact for quadratics), $h_t = \mu_h + \Sigma_h^{1/2}\xi_t$, and at each node *conditions* the inducing inputs on it, per dimension, using the chain's covariance c and the input variance σ^2:

$$h(z_i) \mid h_t \sim \mathcal{N}\Big(\mu_{z_i} + \tfrac{c_i}{\sigma^2}(h_t - \mu_h),\ \sigma_{z_i}^2 - \tfrac{c_i^2}{\sigma^2}\Big), \tag{11}$$

so the kernel at the node is (9) between the certain h_t and the conditioned Gaussian inducing inputs (`_conditioned_kernel`). With $\mathbf{k}_t$ that row,

$$\bar{v} = \frac{1}{q}\sum_t \max(1 - \mathbf{k}_t(K+D)^{-1}\mathbf{k}_t^\top, 0) + \mathrm{Var}_t\left[\mathbf{k}_t\alpha\right], \qquad \text{explained} = 1 - \frac{1}{q}\sum_t \max(\cdot, 0). \tag{12}$$

The clamp is per node because, conditioned on a node, the inducing points sit closer than the expected matrix $\mathbb{E}[K]$ allows and $\mathbf{k}_t(K+D)^{-1}\mathbf{k}_t^\top$ exceeded one at half the draws of a trained folded-section model. The same pass reports, for the prediction (section 3.7), the variance of each node's own residual,

$$\text{own} = \frac{1}{q}\sum_t \max\Big(\max\left(1 - \mathbf{k}_t(K+D)^{-1}\mathbf{k}_t^\top, 0\right) - \|\mathbf{k}_t R\|^2,\ 0\Big). \tag{13}$$

The 64 nodes are summed four at a time (`_NODES_PER_CHUNK`) by `_chunked_sums`: a `while_loop` whose custom gradient recomputes each chunk under a local tape, so one chunk's intermediates are alive at a time in both passes (needed on the GeoScape network; see section 6.4 for its cost).

Nodes that combine kernels of their own (`AdditiveGP`, `MultiStructureGP`) set `_CONDITIONED_SECOND_MOMENT = False` and keep the first moment.

2.4 The walk

`GPWalk` moves each point along an uncertain vector field F (its parent, a GP of the input's size): n steps of length $s = \texttt{step} \cdot \texttt{amp}$. Under the joint rule it is linearized about the mean path and tracks, per point, three sensitivities (`_walk_step`): A to the start, H to the field's normals (its explained part, shared by all points), and r to normals of the point's own (the field's unexplained variance). Each step, with J the expected slope of the field (closed form from the mixture), $g = \mathbf{k}R$ and $\sigma = \sqrt{\max(1 - \mathbf{k}(K+D)^{-1}\mathbf{k}^\top - \|g\|^2, 0)}$:

$$\begin{aligned} &\mu \leftarrow \mu + s(\bar{\mathbf{k}}\alpha + b + \textstyle\sum \mathbb{E}[\nabla\mathbf{k}]R\,H), \\ &A \leftarrow A + sJA, \qquad H \leftarrow H + s(JH + g), \qquad r \leftarrow r + s(Jr + \sigma), \end{aligned} \tag{14}$$

the Stein correction $\mathbb{E}[\nabla\mathbf{k}]RH$ entering from the second step. The walked variance is $\|H\|^2 + \|r\|^2 + A^2\Sigma_0$ (`_walked_variance`) and the covariance between two walked sets $H_x H_y^\top + A_x A_y^\top \Sigma_0$ (`_walked_covariance`). The steps are recomputed in the backward pass (`_recomputed`).

3 Realizations that hold the reported variance

3.1 What the realizations used to carry

A realization was $f^{(s)}(x) = \mathbf{k}(\alpha + R\eta_s) + b$, $\eta_s \sim \mathcal{N}(0, I)$: by (5) it carries only the explained draw $\mathbf{k}RR^\top\mathbf{k}^\top$, leaves out $1 - \mathbf{k}K^{-1}\mathbf{k}^\top$, and at an uncertain input evaluates $\mathbf{k}$ at the parent's *mean*, so it misses the spread of (10) as well. Measured on the folded section (gate 2a, 2000 realizations): 0.59 of the reported variance on the GP on a GP, 0.35 beside the coordinates, 0.45 for the walk; a shallow GP with three experts 0.06.

3.2 Random-feature prior draws

A prior draw of (1) is written with J random Fourier features (`_feature_basis`, `_features`). Each feature picks a component q_j with probability $w_q / \sum w$, a frequency $\omega_j \sim \mathcal{N}(0, 2\omega_{q_j} I_d)$ and a phase $\beta_j \sim U(0, 2\pi)$:

$$\phi_j(x) = \sqrt{\tfrac{2W}{J}} \cos(\omega_j^\top x/\rho + \beta_j), \quad \phi_{J+1}(x) = \sqrt{c}, \qquad f(x) = \phi(x)^\top w, \;\; w \sim \mathcal{N}(0, I_{J+1}), \quad (15)$$

with $W = \sum_q w_q$; then $\mathbb{E}[\phi(x)^\top \phi(y)] = k(x, y)$ because the spectral density of $e^{-\omega\delta^2}$ is $\mathcal{N}(0, 2\omega I)$. At an uncertain inducing input $z \sim \mathcal{N}(\mu_z, V_z)$ the feature is averaged over the input ("damped"): $\mathbb{E}_z[\cos(\omega^\top z/\rho + \beta)] = e^{-\frac{1}{2}\omega^\top \operatorname{diag}(V_z/\rho^2)\omega} \cos(\omega^\top \mu_z/\rho + \beta)$. $J = \max(1024,\, 2\max_e m_e)$, the largest expert of all, active or not (`_feature_count`), so that a prediction by expert draws the same prior. All random numbers are on Philox, named: TensorFlow's `auto_select` picks another generator under XLA.

3.3 Each expert's conditioned draw, from one prior

The user's construction, in three steps:

1. each expert draws its inducing values from its own posterior, coupled to the shared prior: $u_e - \mu_e = M_e \tilde{f}(Z_e)$ with $M_e K_e M_e^\top = S_e$, where $\tilde{f}(Z_e) = L_e G_e^{-1} \Phi_e w$, $\Phi_e = \phi(Z_e)$ and $G_e = \operatorname{chol}(\Phi_e \Phi_e^\top)$ — so $\operatorname{Cov}[\tilde{f}(Z_e)] = K_e$ *exactly*, whatever the features;
2. the experts' conditioned realizations $\mathbf{k}_e K_e^{-1} u_e$ are blended by the weights (7);
3. Matheron's rule adds what the inducing points leave out, $f(x) - \mathbf{k}_e K_e^{-1} \tilde{f}(Z_e)$, which is orthogonal to the conditioned draw in the reproducing-kernel sense.

Any M_e with $M_e K_e M_e^\top = S_e$ makes each expert exact; they differ in how strongly two experts' draws agree. Measured on the folded section's hidden layer at points two experts share, before any rescale (blend over reported):

M_e	blend/reported	step ratio overlap/elsewhere	needs
independent η_e (0.9.3)	0.47	0.98	–
$\operatorname{chol}(S)L^{-1}$ (chosen)	0.84	1.20	L^{-1}
$S^{1/2}K^{-1/2}$	0.87	1.23	$K^{-1/2}$
optimal transport $K^{-\frac{1}{2}}(K^{\frac{1}{2}}SK^{\frac{1}{2}})^{\frac{1}{2}}K^{-\frac{1}{2}}$	0.86	1.17	$K^{-1/2}$
Matheron $D(K+D)^{-1}f(Z) - K(K+D)^{-1}\varepsilon$	0.70	1.16	–
sequential conditioning on earlier experts	0.72	2.04	means pulled off

The Cholesky ratio was chosen because in the realization it needs no inverse of K at all: $\mathbf{k}K^{-1}u = \mathbf{k}\alpha + \mathbf{k}R\,G^{-1}\Phi w$, i.e. today's draw $\mathbf{k}R\eta$ with $\eta = G^{-1}\Phi w$ (`_whitened_prior`); the symmetric and transport maps need $K^{-1/2}$, which a `SelectInput` network's K (conditioned at 10^{18}) would not survive. Matheron's own pseudo-noise form was measured *less* coherent: where the data are dense its noise ε dominates and is independent between experts. The residual is written through the Cholesky as well, $f(x) - (L^{-1}\mathbf{k}^\top)^\top G^{-1}\Phi w$: through K^{-1} a long range put entries of 10^6 in the product, which rounded differently with the batch's shape (5×10^{-8} measured).

3.4 The blend and its exact rescale

Everything random is now linear in w: expert e's deviation at x is $T_e(x)^\top w$ with $T_e = \mathbf{k}_e R_e G_e^{-1}\Phi_e + (\phi(x) - (L_e^{-1}\mathbf{k}_e^\top)^\top G_e^{-1}\Phi_e)$, the first term the conditioned draw, the second the

residual. The blend's deviation is $T^\top w$ with $T = \sum_e w_e T_e$, of variance $\|T\|^2$ exactly, and it is rescaled to the variance the moments report:

$$f^{(s)}(x) = \bar{m}(x) + \varrho(x)\, T(x)^\top w_s, \qquad \varrho = \sqrt{\bar{v}/\|T\|^2} \tag{16}$$

(`_shared_prior_simulate`; `_residual_scale` sets $\varrho = 0$ where $\bar{v} < 10^{-10}$, at an inducing point, where both are rounding). Where one expert holds the weight $\varrho \approx 1$; where they agree the rescale is small. Nodes without a feature kernel keep independent per-expert normals (`_expert_key`) with the blend rescaled to $\sum w_e v_e / \sum w_e^2 v_e$ (`_blend_scale`).

3.5 Through the parent: readers of uncertain inputs

A GP reading an uncertain input draws realization s at the parent's realization s (`_pathwise_simulate`). The parent's realization h_s (plus whatever variance the parent's realizations leave out, location-keyed noise) is standardized against the parent's blended moments, $z_s = (h_s - \mu_h)/\sigma_h$, and each expert reads it in its own view of the parent, $h_{e,s} = \mu_{h,e} + \sigma_{h,e} z_s$ — read whole by every expert, the blended realization put the experts' means off. At $h_{e,s}$ the inducing inputs are conditioned as in (11), $\mathbf{k}_{e,s}$ is the conditioned kernel, the residual uses the features at $h_{e,s}$ against the damped features of the inducing inputs, and the blend is rescaled *draw by draw* to the blended conditional variance $\sum_e w_e \max(1 - \mathbf{k}_{e,s}(K_e + D_e)^{-1}\mathbf{k}_{e,s}^\top, 0)$. The draws are computed a few realizations at a time (`_chunked_map`) with the same recomputing gradient.

3.6 Location-keyed normals

Noise that must not depend on the batch (the parent's left-out variance, training's marginal draws) is drawn per location (`_location_normals`): two 32-bit states are hashed from the coordinates' IEEE bits with the PCG hash (`_pcg32`), mixed with the seed and a CRC of the node's name and purpose, and each column's pair of uniforms becomes normal by Box–Muller. A location's realizations are therefore the same in any batch.

3.7 The prediction integrates the residual (2(b))

The stored realizations carry everything. The *prediction* — the mean of the back-transformed realizations — is instead taken from the same draws without the nodes' own residual (`smooth_draws`), with that residual's variance (13) integrated analytically beside the noise by the likelihood (`predict(..., smooth=, own=)`, `integrated_backward`): the realizations stay rough, the prediction stays smooth, and nothing is averaged by Monte Carlo that can be integrated.

4 Training

For a Gaussian leaf the data term is a quadrature over the latent marginal and needs only the moments, so the second moment enters training directly. For a non-Gaussian leaf (a `GaussianMixture`, `Multiply`) the data term averages over latent samples; since this work each GP node draws those from each location's *full* marginal, $\bar{m} + \sqrt{\bar{v}}\,\xi$ with location-keyed ξ (`marginal_draws`, opened by `_data_log_lik`) — a per-row bound reads each row's marginal only. Previously the samples carried the explained draw alone, a bound too optimistic by the rest. Inside a pass, `once_per_pass` refreshes and propagates each node once, so a node two leaves share is priced once.

5 Where each piece lives

Function (`geoml/latent/network.py`)	Role
`_kernel_components`, `_KERNEL_MIXTURES`	kernel as Gaussians, (1)
`BasicGP.refresh`, `_whitened_root`	K, L, $(K+D)^{-1}$, R, α, (2)–(3)
`expert_kl_terms`	(6)
`_blend`, `get_expert_weights`	(7)
`_chain`, `joint_gain`	chains, (8)
`_expected_kernel`, `_mixture_terms`, `_mixture_sum`	(9)
`_mixture_moments`, `_conditioned_sums`, `_input_nodes`	second moment, (10)–(13)
`_conditioned_kernel`	(11) inside (9)
`_chunked_sums`, `_chunked_map`	loops with recomputing gradients
`GPWalk._walk_step`, `_walked_variance`	(14)
`_feature_basis`, `_features`, `_whitened_prior`	(15), $G^{-1}\Phi$
`_shared_prior_simulate`	certain inputs, (16)
`_pathwise_simulate`	readers of uncertain inputs
`_location_normals`, `_pcg32`	batch-invariant noise
`marginal_draws`, `smooth_draws`, `once_per_pass`	the contexts
`models._predict_raw`, `likelihood.predict`	the smooth pass and (13) integrated

A prediction (`VGPNetwork._predict_raw`) runs, inside one `once_per_pass`: each leaf's `predict` (its `propagate` — moments, chains, the second moment — then `simulate`, which for a reader draws through its parent's realizations and caches the full and smooth outcomes of one draw); then, where any leaf carries an own residual, a second `simulate` under `smooth_draws`, which reads those caches; then each likelihood's `predict` with the smooth realizations and the own variance.

6 Measurements

6.1 The realizations hold the reported variance

Gate 2a draws 2000 realizations at new holes on the folded section and compares their variance with the reported one.

	GP on a GP	beside coordinates	walk	exchange (32 experts)
0.9.3, one expert	0.59	0.35	0.45	–
now, one expert	1.00	1.00	0.99	–
now, three experts	0.93	0.99	0.99	0.994–1.006

On the exchange network the first pathwise draws of this work, with independent noise, had reached 0.82–0.91. The three-expert GP on a GP stays at 0.93 because the experts' mean functions disagree (their spread is 28% of the variance), which a weighted blend of means loses by convexity. Against its exact posterior covariance a shallow GP's realizations have variance 1.00 and a correlation error of 0.023 (median), 0.07 at the 95th percentile.

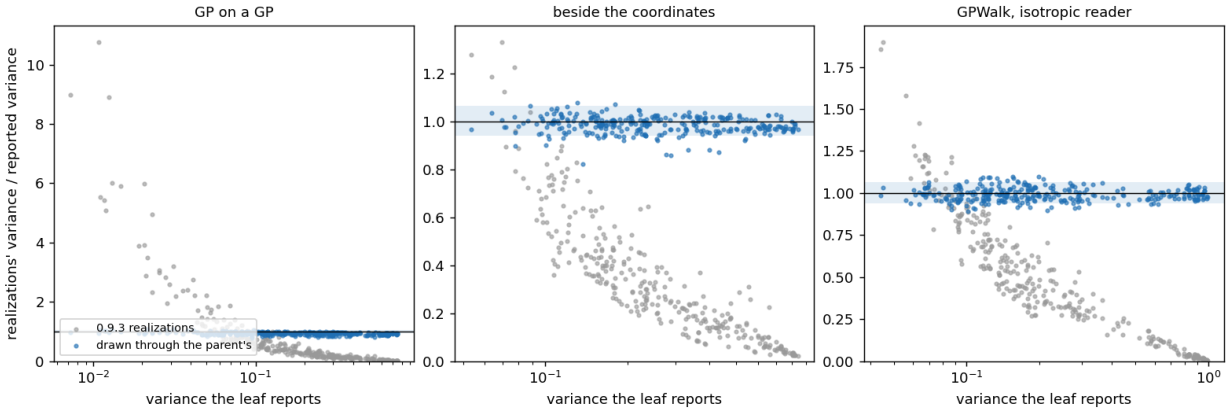


Figure 1: Gate 2a with three experts: realized over reported variance per point.

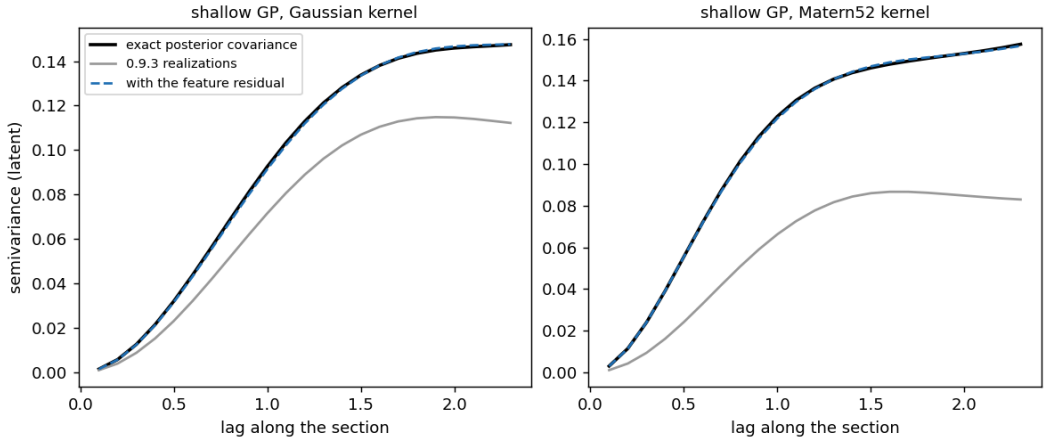


Figure 2: A shallow GP's realizations against its exact posterior covariance.

6.2 Training on the second moment

Three seeds each; Walker scored against the exhaustive field, its fair check being samples of a measurement (its truth carries the nugget the model calls noise).

case	arm	RMSE	meas. CRPS	meas. 90% cov.	time
Walker, GP on a GP	0.9.3 bound	188	102.9	0.99	38 s
	second moment	**160**	**84.9**	**0.89**	504 s
Walker, beside coords	0.9.3 bound	174	91.9	0.96	43 s
	second moment	**161**	**84.8**	**0.89**	720 s

On the folded section the new-hole log score went from -9.6 to -11.2 (moment bound) to -1.9 to -2.4 beside the coordinates. On chapter 16 the rock's log score moved from 0.3224 to 0.3204 on every seed, the metals not at all, for 3.7 times the training time. A sampled warm-up added nothing on any case.

6.3 Two regressions, traced

The folded ore (`non_stationary_test.py`). Three drillholes through a folded ore layer and a walked deep model at transform range 2 folded the ore into one body in the user's 2025-09-08 copy and in commit `c8e4c52`, and gave five separate blobs in every release from v0.3.5 on (three seeds each, 17 versions). Swapping v0.3.5's files one at a time into `c8e4c52` put the cause in `latent.py`; in it, v0.3.5 gave every GP node a trainable bias $b \in [-5, 5]$, the prior mean, which

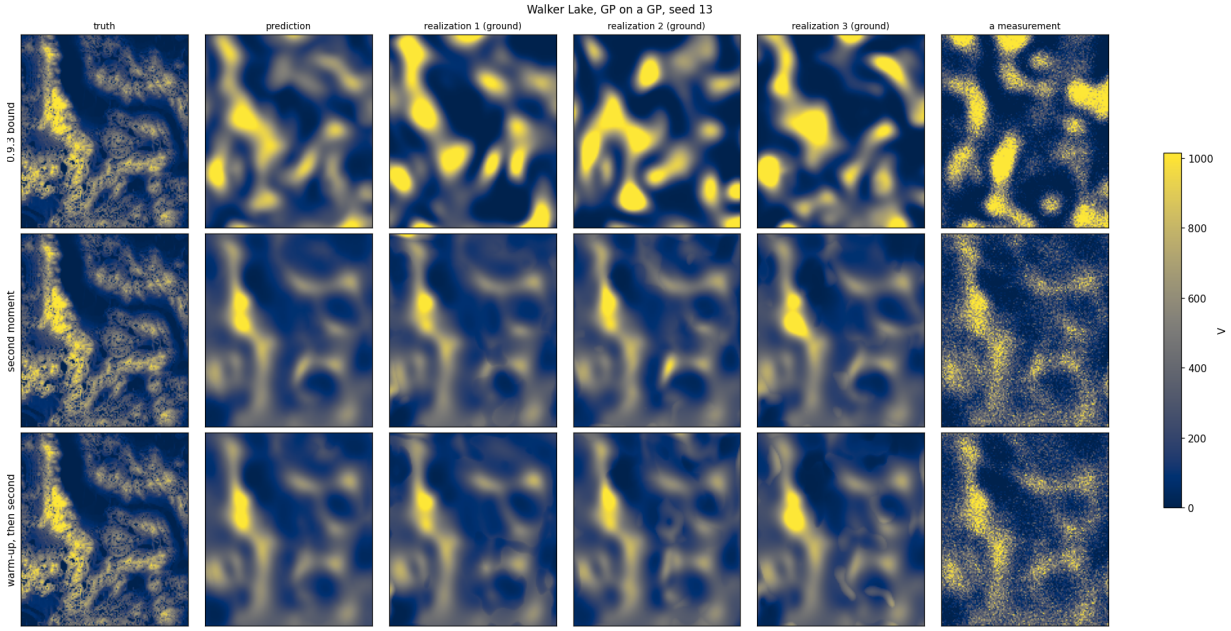


Figure 3: Walker, GP on a GP, seed 13: truth, prediction, three ground realizations and a measurement draw, for the 0.9.3 bound (top), the second moment (middle) and the warm-up (bottom).

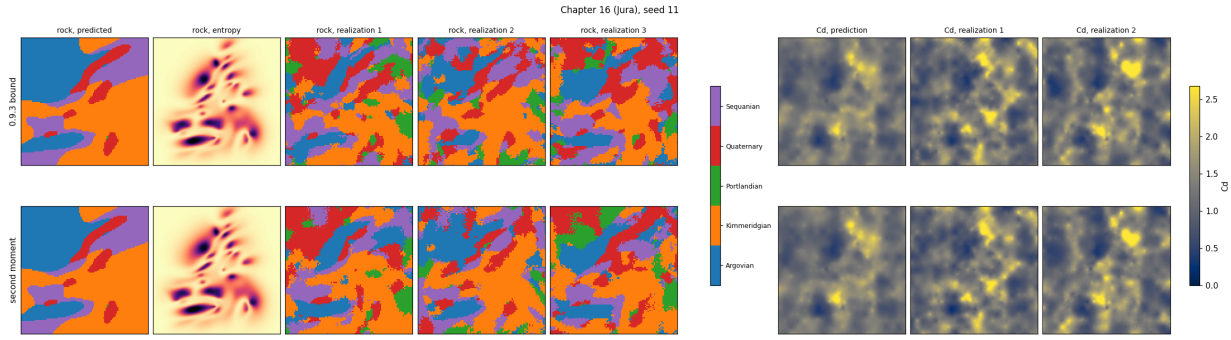


Figure 4: Chapter 16, seed 11: the rock and Cd, the 0.9.3 bound (top) against the second moment (bottom).

had been zero. With zero mean a field returns to zero — the ore/waste boundary — away from the data, and only a fold explains the logs; with a free bias it returns to "waste", and five blobs explain them at a better bound (-48.3 against -51.4). Fixing every bias at zero brings the fold back on v0.3.5, v0.8.8, v0.9.3 and the branch, on all three seeds; at v0.3.5 the bounds equal the old copy's to the digit.

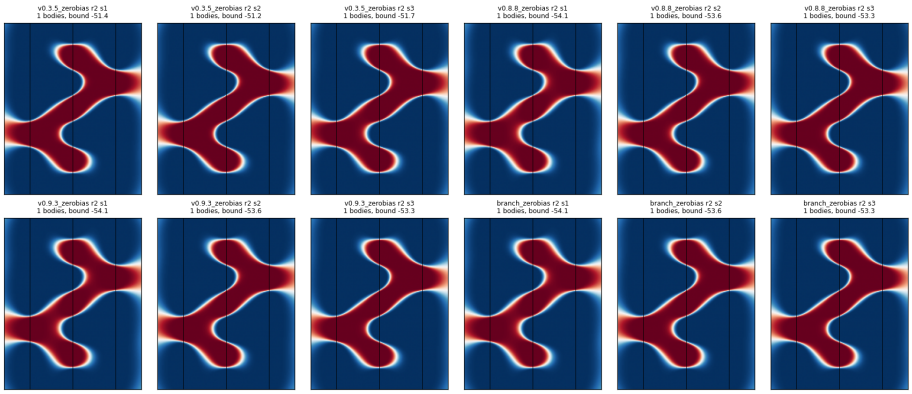


Figure 5: The folded ore with every bias fixed at zero.

Chapter 17. The 0.8.0 chapter's deep model is identical to the bit from 0.8.0 to 0.8.8 and changes at 0.9.0 (bound $-1488 \to -1822$, in-sample agreement $0.983 \to 0.972$). Under `propagation="marginal"` v0.9.3 and the branch reproduce 0.8.x to the last digit: the change is the joint propagation made the default in 0.9.0, for which the chapter was later retuned.

6.4 Where the time goes

A training step of Walker's GP on a GP is 15 ms under the moment bound and 246 ms under the second moment on the CPU. The TF profiler puts 88% of the op time in the conditioned quadrature's loop — 72% its backward pass, which recomputes each chunk — on elementwise operations over $[n, \text{chunk}, m, d]$ arrays. The chunking, not the arithmetic, is much of it:

quadrature, 64 nodes	CPU ms/step	GPU ms/step
4 a chunk, looped (now)	245	130
16 a chunk, looped	172	78
all at once, plain autodiff	147	60

(the bound identical to 10^{-8} in every arm). Recommended: (1) size the chunk by a memory budget and use plain differentiation when everything fits, keeping the recomputing loop for networks whose memory needs it (1.7× on the CPU, 2.2× on the GPU); (2) fewer nodes in low dimension — a 5×5 Gauss–Hermite grid in two dimensions would cost 40% of the 64 Sobol nodes, with weighted sums; not yet measured for accuracy; (3) the prediction is dominated by the same quadrature (the moments); the shared-prior realizations cost under a tenth of it.

References

- Girard, A., Rasmussen, C. E., Quiñonero-Candela, J., Murray-Smith, R. (2003). Gaussian process priors with uncertain inputs. NeurIPS.
- Titsias, M., Lawrence, N. (2010). Bayesian Gaussian process latent variable model. AISTATS.

- Damianou, A., Lawrence, N. (2013). Deep Gaussian processes. AISTATS.
- Salimbeni, H., Deisenroth, M. (2017). Doubly stochastic variational inference for deep Gaussian processes. NeurIPS.
- Rahimi, A., Recht, B. (2007). Random features for large-scale kernel machines. NeurIPS.
- Wilson, J., Borovitskiy, V., Terenin, A., Mostowsky, P., Deisenroth, M. (2020). Efficiently sampling functions from Gaussian process posteriors. ICML.
- Jarzynski, M., Olano, M. (2020). Hash functions for GPU rendering. JCGT 9(3).